

First Programming Task

Type Systems
194.191, 2025W
TU Wien

What to do

Please implement an interpreter for the dynamically typed programming language TIFL specified below. For the implementation you can use any language and all tools you consider as useful. Because of the simplicity of the syntax it is not necessary to use a sophisticated parser generator or another advanced language implementation tool. A simple recursive descent parser will be appropriate: Use one procedure for each syntactic element that deals with this element and then switches to the next procedure (calling that procedure) depending on the next value in the input stream. It is recommended to generate an abstract syntax tree (AST) that contains the parsed program in abstract form (an AST node for each syntactic element) and simplifies interpretation (call methods on AST nodes). Please be careful to check each type constraint while executing a program. An error message shall be issued whenever a type inconsistency is determined. Efficiency is not important.

Specification of TIFL (Typed Interpreted Functional Language)

Syntax. Here is the syntax specification in EBNF. Nonterminals are enclosed in triangular brackets (like `<prog>`), terminals are characters in quotes (like `'abc'`), `|` separates alternatives, parentheses (...) are used for grouping, braces {...} stands for 0, 1 or more repetitions of the contents and square brackets [...] for optional (0 or 1) occurrences.

```
<prog> ::= <def> {',' <def>}. // program is definition sequence
<def> ::= '@' <file_name> // include defs from specified file
        | <tdef> // type definition
        | <vdef>. // value definition
<tdef> ::= <tname> '=' ( <tnom> // specify name of nominal type
                        | <texp> ). // ... or structural type
<tnom> ::= '{' <decl> {',' <decl> } '}' // type of struct
        | '[' <elem> {',' <elem> } ']' // type of union
<decl> ::= <vname> ':' <texp>. // field is of given type
<elem> ::= <vname> '[' <texp> ]. // simple label or typed field
<texp> ::= <tval>
        | <tval> '->' <texp>. // type of a function
<tval> ::= <tname> // name of type
        | '(' <texp> {',' <texp> } ')'. // tuple type or parenth.
<vdef> ::= <vname> '=' <vexp>. // specify name of a value
<vexp> ::= <fval>
        | <vexp> <fval>. // function application
<fval> ::= '\' <decl> '.' <fval> // function definition (lambda)
        | <val>
        | <tname> <spec>. // create or take value of type
<val> ::= <vname> // content of variable (named value)
        | '(' <vexp> {',' <vexp> } ')'. // tuple or parentheses
        | <val> '.' <vname> // access field of tuple or struct
        | <val> '[' <vdef> {',' <vdef> } '[' <vexp> ']' ']' // case
<spec> ::= '[' (<vdef> | <vname>) ']' // union value
        | '{' [<vdef> {',' <vdef>}] '}' // struct field values
```

The nonterminals `<tname>` (type name), `<vname>` (value name) and `<file_name>` are not specified in the syntax. Please find your own, easily distinguishable specifications for type and value names. In this text we use character sequences beginning with uppercase letters as type names and others as value names. Character sequences used in the syntax shall be excluded. Also usual literals and symbols like `+` and `-` are useful as value names because `Int` and `Char` can be represented as union types and functions are values. `<file_name>` can contain anything useful to specify name and location of a file, maybe every character except `'`, `'`.

Unions and enumerations (as well as their types) are quite similar. Our version of unions can also be used for enumerations. For example, `X=[a,x:S,c,y:T]` defines such a type with labels `a`, `x`, `c` and `y`; instances can be created with `X[a]` or `X[c]` (select simple label as enumeration element of `X`) as well as `X[x=b]` or `X[y=d]` (union with a field value).

Syntactically there is no difference between tuples with only one element and simple expressions in parentheses (both for types and values). However, tuples with only one element are not useful in practice. We can regard them as simple types or values directly after parsing.

You can extend the syntax if you want, but it is not necessary to do so. For example, it may be useful to have comments in the language.

Program invocation. A program is a sequence of definitions specifying names of types and values. The language does not support a destructive assignment according to the functional paradigm. We assume the program is contained in a text file (where program parts can be included from further files, e.g., using `@prelude.tifl`). The interpreter is invoked with this text file and a string standing for an expression to be evaluated (conforming to the syntax specified by `<vexp>`). First, the interpreter reads the program. Then, it evaluates the expression in the context of the program and returns the result as output.

Semantics. Comments in the syntax description give ideas of the semantics. Type expressions are clearly separated from value expressions.

In general, a name introduced in a definition (to the left of `=` or `:` as well as simple labels in unions) must be unique, this is, it must not be used for anything else at the same level (same program, struct or union). But, the same name can be used for different things at different levels (e.g., in a program and several structs and several function definitions), where inner definitions hide outer definitions. In the definition of a union type, e.g. `XList=[nil,elem:(X,XList)]`, the type name just being declared can occur in types of fields (recursive type), but only if there is a recursion-free alternative like `nil`. Within function bodies it is allowed to use value names that have not been introduced so far (recursion); however, when functions are executed, all names occurring there must already have been introduced. For each other occurrence of a name the definition of the named entity has to be completed before its name becomes usable.

There are nominal and structural types. Types of structs and unions are nominal, they are equal only if they have been introduced in the same definition. Labels in different struct and union types are different even if they have same names. Types of functions and tuples are structural,

they are equal if they specify the same structure, independent of their names. The same type can have several names: With $T=\{x:A, y:B\}$, $S=T$ the names S and T refer to the same type so that every instance of T is also an instance of S (and the other way around). Function types support currying: A type $R \rightarrow S \rightarrow T$ (equivalent to $R \rightarrow (S \rightarrow T)$) stands for a function taking an argument of R and returning a function that takes an argument S and returns an instance of T , but equivalently for a function taking two arguments of types R and S and returning an instance of T .

Functions are defined as follows: $id=\lambda x:T.x$ where id is a simple identity function returning its argument of type T . Syntax and semantics follow those of λ abstractions in the λ calculus. Currying is easy since $\lambda x:S.\lambda y:T\dots$ corresponds to $\lambda x:S.(\lambda y:T\dots)$ and a function application $f\ a\ b$ corresponds to $(f\ a)\ b$, this is, function f applied to a and then the resulting function applied to b .

To create a value of a nominal type, the type name must be explicitly specified. Using the type definition $T=\{x:A, y:B\}$, we create a struct and assign it to s by $s=T\{x=a, y=b\}$ if a is of type A and b of type B ; then s will be of type T . Values of the same type with same contents are equal. To access fields in structs, we use dot notation: $s.x$ returns a and $s.y$ returns b . No other operations are supported on structs and dot notation is only applicable to structs and tuples (see below). Using $U=[x:A, y:B, z]$ we create a new union by $u=U[x=a]$ specifying only a value for one label or selecting a simple label $v=U[z]$. To access the contents of a union, we need a case expression that specifies a function for each field label and a value for each simple label in the type: $u[x=\lambda x:A.x, y=\lambda y:B.c, z=d]$ returns a because the label in u is x and the function for x is just the identity function on type A . The result of $v[x=\lambda x:A.x, y=\lambda y:B.c, z=d]$ is d because v contains z , and we expect d to be of type A because the result type shall be independent of the label. Although it is difficult, you should at least try to ensure that all function results (for field labels) and values (for simple labels) are of the same type and report an error otherwise. Case expressions can contain a default clause (after $|$) standing for all labels not mentioned before as in $u[x=\lambda x:A.x, z=d|\lambda y:B]$ (with the same semantics as the above example). The default clause is usable only if all remaining labels require an expression of the same type. No other operations are supported on unions. As an example, we use unions to deal with Boolean values: $Bool=[true, false]$ defines the type, $Bool[true]$ and $Bool[false]$ are the instances. Let t be one of these values. A case expression offers conditional execution: $t[true=a|b]$ results in a for $Bool[true]$ and b for $Bool[false]$. For any expression c returning a value of type $Bool$ we can use $(c)[true=a|b]$ or equivalently $(c)[false=b|a]$ instead of $if(c)then(a)else(b)$ in other languages.

A tuple is just a sequence of values and the corresponding tuple type is the sequence of types of these values. Tuple types are structural, their names are not relevant for distinguishing between types. For example, with $T=\{x:A, y:B\}$ and $s=T\{x=a, y=b\}$, the tuple $(s.x, s.y)$ is of type (A, B) and is equal to (a, b) ; there is no relation to T and s anymore. Elements of a tuple are accessible using dot notation, where numbers $1, 2, \dots$ are used as labels. For example, $(a, b).1$ is a , $(a, b).2$ is b and $(a, b).3$ is an error. There is no other way to access elements in tuples.

Basics defined in TIFL.

The language does not directly support basic operations like integer operations and comparisons. It is possible to define much functionality directly in the language (as examples):

```
Bool = [true,false],
Char = ['a','b','c','d'],
Int = [0,i:Int,nan],
String = ["",elem:(Char,String)],
true = Bool[true], false = Bool[false],
'a'=Char['a'], 'b'=Char['b'], 'c'=Char['c'], 'd'=Char['d'],
0 = Int[0], 1 = Int[i=0], 2 = Int[i=1], 3 = Int[i=2],
"" = String[""],
validNum = \x:Int.x[0=true, i=\i:Int.true, nan=false],
decr = \x:Int.x[nan=x, 0=Int[nan], i=\i:Int.i],
incr = \x:Int.x[nan=x, 0=1, i=\v:Int.Int[i=x]],
+ = \x:Int.\y:Int.x[nan=x, 0=y,
                    i=\v:Int.y[nan=y, 0=x, i=\i:Int.+ v (incr y)]],
< = \x:Int.\y:Int.y[i=\b:Int.x[0=true,
                                nan=false,
                                i=\a:Int.< a b]
                    | false],
min = \x:Int.\y.Int(< x y)[true=x|y],    // nan not handled!
++ = \s:String.\t:String.s[""=t
    | \p:(Char,String).String(elem=(p.1, ++ p.2 t))],
...
```

Definitions like `0=Int[0]` make names (0) easily accessible as values of specified types (Int) and ensure that these names cannot be used for something else at the global scope. Obviously definitions as in the above example cause programs to run inefficiently and are feasible only if value ranges are relatively small. Our goal in this programming task is to find out how to deal with nontrivial explicit types in a dynamic language. Having this purpose in mind it is reasonable to keep value ranges small and ignore run time efficiency.

Please write a file `prelude.tifl` with definitions to be included in general purpose programs. There shall exist predefined types for at least Boolean values, characters, integers (including negative values) and strings as well as usual functions on them, including logic operations, basic arithmetics, comparisons, basic string operations and so on. All functions shall be designed such that it is rather easy to write programs not suffering from type errors. For example, if there is a function returning the *n*th character in a string, it is necessary to clearly define a value to be returned in the case of an insufficient number of characters, and there must also be a simple way to find out if the string has enough characters.

Another purpose of this file and other program files is to provide code for testing the interpreter. Please provide test cases for all parts of the language showing how they can be successfully used. But, there is also a need for test cases showing that (nontrivial) type errors are detected; corresponding program runs shall result in error messages.